



Diehl AKO Stiftung & Co. KG Photovoltaics

Requirements Specification PLATINUM EasyConnect

Manual

Approved

Version : 1.0.7
State : Valid
Date : 2011-10-20
Author : Produkt Management
Manager : ---

Diehl AKO Stiftung & Co. KG
CCG2PDVS
Pfannerstrasse 75
D-88239 Wangen im Allgäu

phone +49 (0)7522/73-0
fax +49 (0)7522/73-250
e-mail info@diehl-controls.com
web www.diehl.com/photovoltaics



Change History

Version	Chapter	Change Description	Author	Date
1.0.0	All	Preparation of Document	Produkt Management	2011-05-12
1.0.1	3.2.2	Added Power Reduction Value	rieft	2011-05-20
1.0.2	3.2.4	Detailed event transmission	rieft	2011-05-26
1.0.3	3.2.1	Added AC Power value	rieft	2011-06-08
1.0.4	2.4	Corrected Serial Number	rieft	2011-07-07
1.0.5	4	Added Handling with different Objects	rieft	2011-10-11
1.0.6	4 5.5	Protocol Objects Support Matrix Inverter	rieft	2011-10-18



Contents

	Change History	2
1	About this Document.....	6
1.1	Intention	6
1.1	References.....	6
1.2	Hints.....	6
1.3	Archiving	6
1.4	Referenced Documents.....	6
1.5	Definitions/Abbreviations	7
2	Functional Description of the PLATINUM EasyConnect Protocol	8
2.1	Physical interface	8
2.2	Structure of a data telegram	8
2.3	Addressing	9
2.4	EasyConnect Master Serial number	10
2.5	Telegram type	11
2.6	Packet number	11
2.7	Data	12
2.7.1	Checksum CRC	12
2.8	ACK telegram (Acknowledge)	12
2.9	NAK telegram (No Acknowledge)	12
2.10	ACKBSY telegram (Acknowledge Busy).....	13
2.11	Bus free check time	13
2.11.1	Function.....	13
2.12	Collisions.....	13
2.12.1	Collision detection.....	13
2.12.2	Collision treatment.....	14
2.12.3	Jam sequence	14
2.13	Program flow diagram Telegram Send	15
3	Protocol description of the PLATINUM EasyConnect Protocol	16
3.1	Object-orientierted data exchange.....	16
3.1.1	String object.....	16
3.2	Reading data.....	17
3.2.1	Telegram „Read Info String“ and response	17
3.2.2	Telegram „Read Standard Data String“ and response	18



3.2.3	Telegram „Read Detailed Data String“ and response.....	19
3.2.4	Telegram „Read Events String“ and response	21
4	Access to Specific Data Objects using the Standard PLATINUM Protocol	22
4.1	Available Data Objects usable.....	22
4.2	Reading data.....	24
4.2.1	Telegram „Read Data“ and response.....	24
5	Appendix A	25
5.1	Event Codes.....	25
5.2	Event Classes	25
5.3	Event Sources.....	25
5.4	State Codes	25
5.5	Support Matrix for the EasyConnect Objects.....	26

Tables

Table 1: Definitions and Abbreviations	7
Table 2: Transmission parameters.....	8
Table 3: Basic telegram structure to be used for communication	8
Table 4: Byte structure of the serial number	10
Table 5: Content data of the serial number	10
Table 6: Byte structure of the telegram type.....	11
Table 7: List of all relevant telegram types	11
Table 8: Byte structure of the packet number.....	11
Table 9: ACK telegram structure	12
Table 10: NAK telegram structure	12
Table 11: ACKBSY telegram structure	13
Table 12: List of all free check times	13
Table 13: Byte structure of a JAM sequence.....	14
Table 14: List of all relevant object IDs.....	16
Table 15: Example of a string object	16
Table 16: Data request telegram to request the „Info String“	17
Table 17: Data response telegram to the „Info String“ request	17
Table 18: Data content of the „Info String“	17
Table 19: Data request telegram to request the „Data String“	18
Table 20: Data response telegram to the „Data String“ request.....	18

Table 21: Data content of the “Data String”	18
Table 22: Data request telegram to request the “Detailed Data String”	19
Table 23: Data response telegram to the “Detailed Data String” request.....	19
Table 24: Data content of the “Detailed Data String”	19
Table 25: Data request telegram to request the “Events String”	21
Table 26: Data response telegram to the “Events String” request	21
Table 27: Data content of the “Events String”	21
Table 28: List of the relevant object IDs	22
Table 29: Data descriptions of the relevant object IDs	22
Table 30: Presence of the relevant object IDs in the different inverter types	23
Table 31: Data request telegram.....	24
Table 32: Data response telegram	24
Table 33: Support Matrix.....	26

Figures

Figure 1: CRC checksum calculation used to secure communication.....	12
Figure 2: Program flow to send a telegram.....	15

1 About this Document

1.1 Intention

[Explanation of the general intention of the present document]

This document describes the functionality of the DiehlAKO PLATINUM EasyConnect Protocol. The EasyConnect protocol is an extract of the PLATINUM Bus Protocol and intends to ease the retrieval of data by external users.

1.1 References

[Indication to the presence of a reference list, to be replaced with -na- if not used]

- na -

1.2 Hints

[Explanation of hint abbreviations, like TODO, to be replaced with -na- if not used]

- na -

1.3 Archiving

[Explanation of the proceeding to document management]

- na -

1.4 Referenced Documents

[Listing of documents referenced in the present document, to be replaced with -na- if not used]

[Ref 1] Protokollspezifikation EIA485, V 5.0.7, 16.03.2011:

V:\PV\Development\Monitoring\D10637_EIA485_Bus\Documents\External\Protokollspezifikation_EIA-485_v5.0.7.doc

1.5 Definitions/Abbreviations

Definition	Abbreviation	Explanation
Originator	DA	Diehl AKO Stiftung
PLATINUM Bus Protocol	PBP	Communication Protocol of the PLATINUM Products
PLATINUM EasyConnect Protocol	PEC	Extract of the PBP
Diehl AKO Inverter	S	Single phase inverter model with transformer
Diehl AKO Inverter	TL	Single phase inverter model without transformer
Diehl AKO Inverter	3xTL	Three phase inverter model without transformer
Diehl AKO Inverter	TL3	Three phase inverter model without transformer
Diehl AKO Inverter	TLD	Single phase inverter model without transformer
Diehl AKO Inverter	TLxD	Three phase inverter model without transformer

Table 1: Definitions and Abbreviations

2 Functional Description of the PLATINUM EasyConnect Protocol

The HAL is the part of the driver that creates/sends and receives/decodes the telegrams. The checking for collisions and the respective corrections are done in the HAL as well.

2.1 Physical interface

The data transmission is effected via a RS485 interface with the following transmission parameters:

Parameter	Value
Baud rate	19200 baud
Data bits	8
Parity	none
Stop bits	1

Table 2: Transmission parameters

2.2 Structure of a data telegram

Every telegram has the following format:

Offset	Length	Content	Description
0	1	0x16	SYN
1	1	0x10	DLE
2	1	0x02	STX
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	...	Telegram type
12	1	...	Packet number
13	N	...	Data
13+N	1	0x10	DLE
14+N	1	0x03	ETX
15+N	2	...	CRC

Table 3: Basic telegram structure to be used for communication

The beginning of a data telegram is indicated by the SYN character sequence, followed by DLE and STX. This makes sure that the receivers can synchronize to the beginning of the telegram at any time, even in case of malfunctions.

Should a DLE character sequence (0x10) appear when sending data from the grey area, it will be put out twice. When receiving, every second of several DLE characters in a row must be eliminated.

The 16-bit check sum is calculated over the complete telegram (from Offset 3 to Offset 12+N).

As a rule, data words (16 and 32 bit) are transmitted in Motorola format (high-byte first).

2.3 Addressing

The source and target serial numbers are each 32 bit wide, the number includes the device's serial number. Since two devices are not allowed to have the same serial number, every serial number may only be assigned once.

Thus, a maximum of $2^{32}=4,294,967,296$ distinguishable devices is possible.

Broadcast telegrams that are used for the configuration and the network search have a special characteristic: These telegrams are addressed to several or all participants on the bus. A broadcast telegram shows the value 0xFFFFFFFF in the target address field.

2.4 EasyConnect Master Serial number

As mentioned before, every bus participant needs a unique serial number to ensure a proper addressing of the telegrams on the bus.

The EasyConnect master device gets a fixed serial number described here to make things easier.

The serial number consists of a block of 4 bytes:

Byte 1 is the MSB and Byte 4 the LSB.

Byte 1				Byte 2				Byte 3				Byte 4																			
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P1				P2				Line				Year				Month				Day				Seq. number							

Table 4: Byte structure of the serial number

Field	Bit width	Explanation
P1	4 Bit	Basic type of article number Communication devices = 4
P2	4 Bit	Further detailing
L: Produktionslinie	3 Bit	External suppliers : L = 0
J: Jahr	5 Bit	Year since 2000, example: 6 stands for 2006
M: Monat	4 Bit	Months (1 .. 12)
T: Tag	5 Bit	Day (1..31)
N: Lfd. Nummer	7 Bit	not used, must be 0.

Table 5: Content data of the serial number

The serial number 0xFFFFFFFF is an exception. This serial number indicated broadcast telegrams. Broadcast telegrams are addressed to all bus participants and are not allowed to be answered.

The serial number for the minibus device asking for data is for the ease of use fixed to:

4120.JJMMTT000

CAUTION: The usage of two devices using this fixed ID, yields to malfunction of the bus communication and must be avoided.

2.5 Telegram type

By means of the telegram type, one can distinguish whether the telegram is an inquiry or a response telegram and determine the type of the data the telegram contains.

Bit meaning	
7	1=inquiry telegram, 0=response telegram
6	Telegram type 0 .. 127
5	
4	
3	
2	
1	
0	

Table 6: Byte structure of the telegram type

Bit 7 of the telegram type is used to distinguish inquiry and response telegrams. This leaves 7 bits for coding the telegram sort (128 telegram types).

Constant	Value	Telegram type
PTYPE_ACK	0x7C	ACK (Acknowledge, data accepted)
	0xFB	Reserv. (PTYPE_NAK with set bit 7)
PTYPE_NAK	0x7B	NAK (No Acknowledge, data refused)
	0xFA	Reserv. (PTYPE_ACKBSY with set bit 7)
PTYPE_ACKBSY	0x7A	ACKBSY (Acknowledge Busy, no response possible at the moment)
PTYPE_REQUEST_DATA_RQ	0xF2	Read data of participant
PTYPE_REQUEST_DATA_AN	0x72	Response Read data of participant

Table 7: List of all relevant telegram types

2.6 Packet number

The packet number indicates the number of the following packets if more data have to be transmitted than can be placed in the data field. The PEC will always send one answer telegram, therefore there is no need to implement this handling.

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
1: first/last packet	0						

Table 8: Byte structure of the packet number

2.7 Data

Technically, the transmittable amount of data in the data field is unlimited. In order to limit the size of the reception and sending buffers (RAM requirement), the data field size is limited to 64 bytes.

2.7.1 Checksum CRC

The CRC is calculated with the following algorithm or by using a table.

```
unsigned short calcChecksum(const unsigned char *buffer, const int
length)
{
    unsigned short usCRC;
    usCRC = 0xffff;
    for (int i=0; i<length; i++)
    {
        usCRC ^= (unsigned short)buffer[i] << 8;
        for (int j=0; j<8; j++)
            usCRC = usCRC & 0x8000 ? (usCRC << 1) ^ 0x1021 : usCRC << 1;
    }
    return usCRC ^ 0xffff;
}
```

Figure 1: CRC checksum calculation used to secure communication

2.8 ACK telegram (Acknowledge)

A participant responds to a successfully received telegram with a data telegram or an ACK telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x7C	Telegram type = PTYPE_ACK

Table 9: ACK telegram structure

2.9 NAK telegram (No Acknowledge)

A participant responds to a successfully received, yet refused telegram with a NAK telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x7B	Telegram type = PTYPE_NAK

Table 10: NAK telegram structure

2.10 ACKBSY telegram (Acknowledge Busy)

If a participant is temporarily unable to respond to a data request, it responds with an ACKBSY telegram. Then the participant will repeat its data requests until it receives a response other than ACKBSY.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x7A	Telegram type = PTYPE_ACKBSY

Table 11: ACKBSY telegram structure

2.11 Bus free check time

2.11.1 Function

Before inquiry telegrams can be sent on the bus, one has to wait for the bus to be free. For this purpose, a timer, which otherwise is running normally, is reset with every byte received by the bus. Sending inquiry telegrams on the bus is only allowed if at least the effective free check time for the respective telegram type has elapsed. The minimum free check time is the time required for transmitting one data byte; for 19,200 baud, this is about 520µs. In order to enable a collision treatment also in systems with limited real time capability (e.g. PCs using the Microsoft Windows operating system), the time intervals are multiples of 10ms.

Constant	Value	Description
IFS_LOST_ARBR	20	Free check time after lost arbitration
IFS_COLLISION_0	20	Free check time in ms after collision, bit value=0
IFS_COLLISION_1	30	Free check time in ms after collision, bit value=1
IFS_REQUEST	40	Free check time in ms before an inquiry telegram

Table 12: List of all free check times

2.12 Collisions

2.12.1 Collision detection

Collisions can only occur when sending inquiry telegrams. Thus, only telegrams not preceded by an inquiry are checked for collisions. After sending every byte, the receive path is monitored for the echo of the character nsequence. If a wrong echo or no echo at all is detected, this is considered as a collision. In this case, the participant sends some bytes (jam sequence, see 2.12.3), so that the participant(s) which had tried to send data at the same time will unerringly detect a collision.

2.12.2 Collision treatment

After a collision has occurred, the affected participants will try to send the inquiry telegram again. The time between the collision and the new attempt depends on the serial number (IFS time, Inter Frame Space). Thus, repeated collisions of participants are resolved after 32 attempts at the latest since one of the participants will definitely win the collision treatment.

The IFS time is set by the 32-bit serial number in dependence on the bit value, beginning with bit 0 (LSB):

IFS_COLLISION_0 = 20ms, if current address bit of serial number was 0

IFS_COLLISION_1 = 30ms, if current address bit of serial number was 1

This guarantees that the participant with the lower serial number will win the Arbitrierung after 32 repeated attempts at the latest and can send its telegram. In the worst case, it will take $32 * (10\text{ms} + 30\text{ms}) = 1.28$ seconds to resolve a collision between two participants.

2.12.3 Jam sequence

If a participant detects a collision it will send a so-called „jam sequence“. This is a sequence of bytes that collides with the data of the other participant(s) again.

Thus, the other bus participants that are trying to send data at the same time can definitely detect the collision as well.

This sequence must be a byte sequence that does not normally occur in a telegram. This guarantees that the jam sequence differs from the collision partner's data flow by at least 1 bit, so that a collision will be definitely detected.

Byte 0	Byte 1	Byte 2
0x00	0x10	0x00

Table 13: Byte structure of a JAM sequence

According to the protocol specification, this sequence can never occur in a telegram, since the DLE character sequence (0x10) is always followed by an STX (0x02), an ETX (0x03) or another DLE, but never by the byte 0x00.

The initial 0x00 before the DLE in the jam sequence makes sure that any 0x10 sent before the character sequence 0x10 will collide.

2.13 Program flow diagram Telegram Send

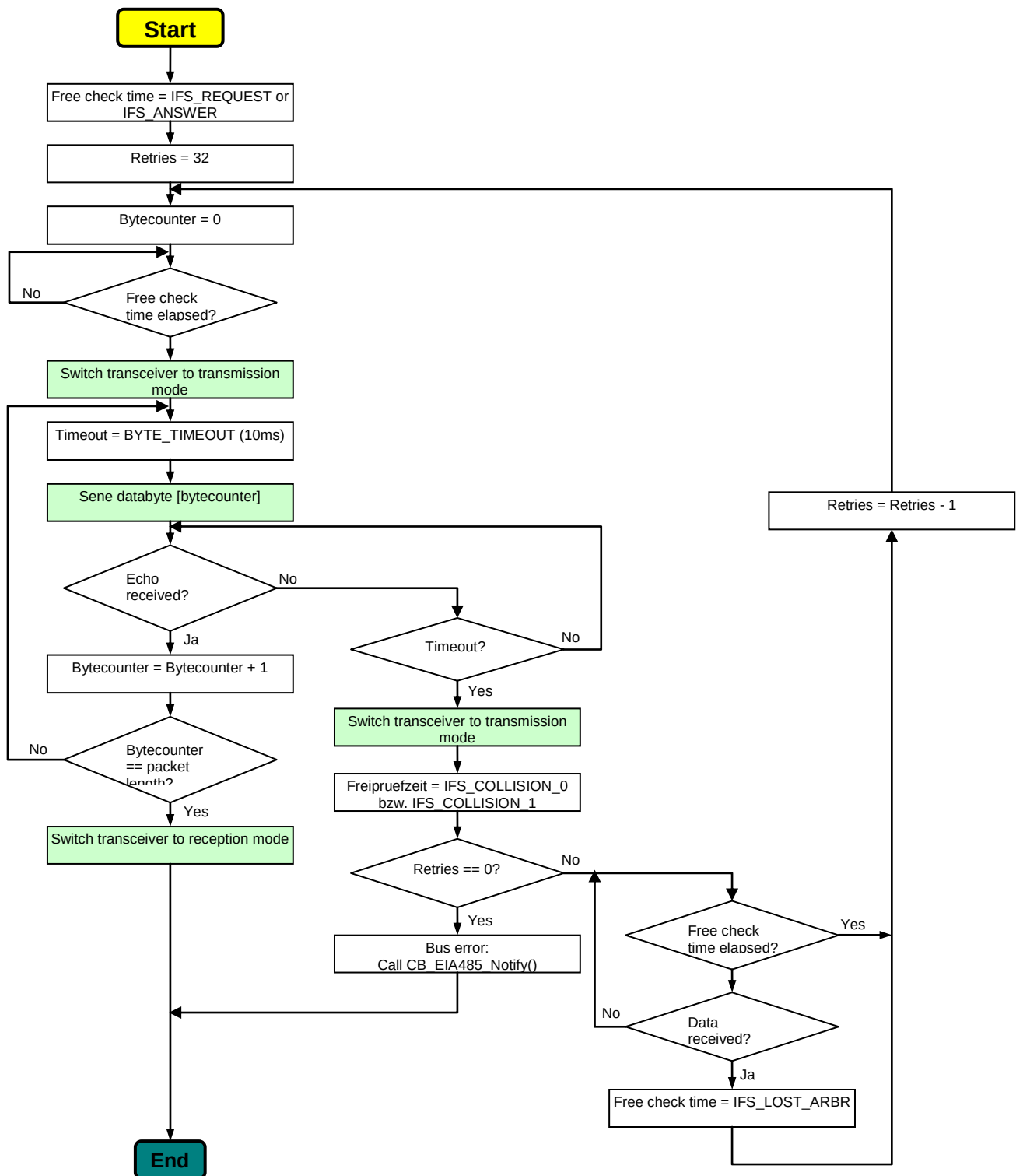


Figure 2: Program flow to send a telegram

3 Protocol description of the PLATINUM EasyConnect Protocol

3.1 Object-orientated data exchange

The Data Layer is responsible for the object-oriented data communication between the individual bus participants. The data are filed in objects.

Every data object has a description (data description object) that provides information on type, sort, accuracy and unit of the data object.

Due to the ease of use of the PEC the data is always sent as readable, volatile string object without precision.

Object	Code	Description
OTYPE_MINI_INFO	0x0090	String for EasyConnectprotocol containing setup data like the type of device
OTYPE_MINI_DATA	0x0091	String for EasyConnect protocol containing standard actual measurement data of the device
OTYPE_MINI_DATA_DET	0x0092	String for EasyConnect protocol containing detailed actual measurement data of the device
OTYPE_MINI_EVENTS	0x0093	String for EasyConnect protocol containing the last valid event codes of the device

Table 14: List of all relevant object IDs

3.1.1 String object

A string object contains a character sequence. The first byte indicates the length of the character sequence.

String length	,0'	,1'	,2'	,,1'	,4'	,0'	,2'	,0'	,,1'
0x08	0x4C	0x65	0x69	0x73	0x74	0x75	0x6E	0x67	0x00

Table 15: Example of a string object

The string objects used for the PEC have a similar structure to a standards csv row in a csv file. The data strings are separated by a semicolon.

If any data content is not deliverable by the device the data is requested from the corresponding field in the string remains empty.

3.2 Reading data

3.2.1 Telegram „Read Info String“ and response

This telegram is used to read the “Info String” of a participant.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0xF2	Telegram type = PTYPE_REQUEST_DATA_RQ
12	1	0x80	Packet number
13	2	0x0090	OTYPE_MINI_INFO

Table 16: Data request telegram to request the “Info String”

The respective participant responds to the inquiry with the following telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x72	Telegram type = PTYPE_REQUEST_DATA_AN
12	1	0x80	Packet number
13	n	...	Requested data

Table 17: Data response telegram to the “Info String” request

The required data is transmitted in the data field of the telegram. The data is delivered as a semicolon separated string of data fields.

Data	Example	Unit	Max Size	Description
Typcode	84C6		4	Typcode of the inverter
Type	6300TLD		20	Inverter type as string
DC Power max	6300	W	7	Maximum DC power of the device
AC Power max	6200	W	7	Maximum AC power of the device
Inverter Number	1		2	Network number of the device
Feeding Phase	3		1	Feeding phase of the device (4 for all)
Country-/Gridcode	DE0		4	Grid code active in the device

Table 18: Data content of the “Info String”

Maximum string length:

46 Bytes + 7 Semicolons = 52 Bytes

String representation:

8086;6300TLD;6300;6200;1;;DE0;

3.2.2 Telegram „Read Standard Data String“ and response

This telegram is used to read the “Measurement String” of a participant.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0xF2	Telegram type = PTYPE_REQUEST_DATA_RQ
12	1	0x80	Packet number
13	2	0x0091	OTYPE_MINI_DATA

Table 19: Data request telegram to request the “Data String”

The respective participant responds to the inquiry with the following telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x72	Telegram type = PTYPE_REQUEST_DATA_AN
12	1	0x80	Packet number
13	n	...	Requested data

Table 20: Data response telegram to the “Data String” request

The required data is transmitted in the data field of the telegram. The data is delivered as a semicolon separated string of data fields.

Data	Example	Unit	Max Size	Description
AC Power eff.	4174	W	7	Actual effective AC power of the device
DC Power eff.	4300	W	7	Actual effective DC power of the device
Power react.	2000	VAR	7	Actual reactive power of the device
Phase Shift Angle	42,77	°	5	Actual phase shift angle of the device
Power Reduction	80	%	3	Actual commanded power reduction
Energy today	4587	kWh	7	Energy generated for the actual day
Energy total	1230500	kWh	11	Energy generated during device lifetime
Temperature	25,2	°C	4	Temperature of the device
Inverter State	12		3	Operating state of the device

Table 21: Data content of the “Data String”

Maximum string length including 8 semicolons:

54 Bytes + 9 Semicolons = 63 Bytes

String representation:

4174;4300;2000;42,77;4587;1230500;25;12;

3.2.3 Telegram „Read Detailed Data String“ and response

This telegram is used to read the “Measurement String” of a participant.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0xF2	Telegram type = PTYPE_REQUEST_DATA_RQ
12	1	0x80	Packet number
13	2	0x0092	OTYPE_MINI_DATA_DET

Table 22: Data request telegram to request the “Detailed Data String”

The respective participant responds to the inquiry with the following telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x72	Telegram type = PTYPE_REQUEST_DATA_AN
12	1	0x80	Packet number
13	n	...	Requested data

Table 23: Data response telegram to the “Detailed Data String” request

The required data is transmitted in the data field of the telegram. The data is delivered as a semicolon separated string of data fields.

Data	Example	Unit	Max Size	Description
AC Current 1	6,8	A	5	Apparent AC current of phase 1
AC Current 2	6,7	A	5	Apparent AC current of phase 2
AC Current 3	6,5	A	5	Apparent AC current of phase 3
AC Voltage 1	220	V	3	AC voltage of phase 1
AC Voltage 2	222	V	3	AC voltage of phase 2
AC Voltage 3	221	V	3	AC voltage of phase 3
DC Current 1	6,9	A	5	DC current of input 1
DC Current 2	6,9	A	5	DC current of input 2
DC Current 3	6,9	A	5	DC current of input 3
DC Voltage 1	600	V	4	DC voltage of input 1
DC Voltage 2	600	V	4	DC voltage of input 2
DC Voltage 3	600	V	4	DC voltage of input 3

Table 24: Data content of the “Detailed Data String”



Maximum string length:

51 Bytes + 12 Semicolons = 63 Bytes

String representation:

String containing data of an inverter feeding on three phases:

6,8;6,7;6,5;220;222;221;6,9;;;600;;;

String containing data of an inverter using three MPP trackers and three feeding phases:

6,8;6,7;6,5;220;222;221;6,9;6,9;6,9;600;600;600;

String containing data of an inverter using one MPP tracker and one feeding phase:

6,8;;;220;;;6,9;;;600;;;

3.2.4 Telegram „Read Events String“ and response

This telegram is used to read the “Events String” of a participant.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0xF2	Telegram type = PTYPE_REQUEST_DATA_RQ
12	1	0x80	Packet number
13	2	0x0093	OTYPE_MINI_EVENTS

Table 25: Data request telegram to request the “Events String”

The respective participant responds to the inquiry with the following telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x72	Telegram type = PTYPE_REQUEST_DATA_AN
12	1	0x80	Packet number
13	n	...	Requested data

Table 26: Data response telegram to the “Events String” request

The required data is transmitted in the data field of the telegram. The data is delivered as a semicolon separated string of data fields.

Data	Example	Unit	Max Size	Description
State	12		3	Operating state of the device
Event Code	402		4	Actual event code of the device
Event Detail	615		4	More detailed event code of the device
Event Class	10		2	Class of the actual event code
Event Source	1		2	Source of the actual event code

Table 27: Data content of the “Events String”

Maximum string length:

15 Bytes + 5 Semicolons = 20 Bytes

String representation:

12;4020;615;10;1;

Caution: The Event Detail provides a more detailed event code available in some inverters. So the TL series will provide some over and underrun information with this code and the TL3 and C series will provide an explicit code different from the Event Code value.

4 Access to Specific Data Objects using the Standard PLATINUM Protocol

4.1 Available Data Objects usable

Object	Code	Description
OTYPE_TIME	0x0008	Actual time of the inverter
OTYPE_STATUS	0x000B	Actual operating state of the inverter
OTYPE_DC_POWER	0x0017	Actual generator power of the inverter
OTYPE_AC_POWER	0x0020	Actual feeding power of the inverter
OTYPE_ENERGY_TODAY	0x0024	Fed in energy today
OTYPE_ENERGY_TOTAL	0x0025	Fed in energy total
OTYPE_DEVICE_TYPE	0x0043	Product identifier, e.g. "7200 TLD"
OTYPE_DEVICE_POWERCLASS	0x0045	Power class of the inverter
OTYPE_PHASESHIFTANGLE_ACTUAL	0x0071	Actual adjusted reactive power of the inverter
OTYPE_REACTIVEPOWER_ACTUAL	0x0076	Actual adjusted phase shift angle of the inverter
OTYPE_POWER_REDUCE_REL	0x0082	Value of the actual relative power reduction
OTYPE_TEMPERATURE1	0x100E	Temperature of the interior space of the inverter

Table 28: List of the relevant object IDs

Object	Length	Type	Precision/Multiplier	Unit
OTYPE_TIME	4 Byte	unsigned	1	---
OTYPE_STATUS	1 Byte	unsigned	1	---
OTYPE_DC_POWER	2 Byte	unsigned	1 (10 for TL3)	W
OTYPE_AC_POWER	2 Byte	unsigned	1 (10 for TL3)	W
OTYPE_ENERGY_TODAY	4 Byte	unsigned	1 (10 for TL3)	Wh
OTYPE_ENERGY_TOTAL	4 Byte	unsigned	1 (10 for TL3)	Wh
OTYPE_DEVICE_TYPE	String	---	---	---
OTYPE_DEVICE_POWERCLASS	String	---	---	---
OTYPE_PHASESHIFTANGLE_ACTUAL	2 Byte	signed	0,01	---
OTYPE_REACTIVEPOWER_ACTUAL	2 Byte	signed	1	%
OTYPE_POWER_REDUCE_REL	1 Byte	unsigned	1 (0,1 for TL3)	%
OTYPE_TEMPERATURE1	2 Byte	signed	0,1	°C

Table 29: Data descriptions of the relevant object IDs

Object	S	TL	3xTL(Large TL)	TL3	TLD	TLxD
OTYPE_TIME	X	X	X	X	X	X
OTYPE_STATUS	X	X	X	X	X	X

OTYPE_DC_POWER	X	X	X	X	X	X
OTYPE_AC_POWER	X	X	X	X	X	X
OTYPE_ENERGY_TODAY	X	X	X	X	X	X
OTYPE_ENERGY_TOTAL	X	X	X	X	X	X
OTYPE_DEVICE_TYPE	X	X	X	X	X	X
OTYPE_DEVICE_POWERCLASS	X	X	X	X	X	X
OTYPE_PHASESHIFTANGLE_ACTUAL	---	---	---	X	X	X
OTYPE_REACTIVEPOWER_ACTUAL	---	---	---	---	X	X
OTYPE_POWER_REDUCE_REL	X	X	X	X	X	X
OTYPE_TEMPERATURE1	X	X	X	X	X	X

Table 30: Presence of the relevant object IDs in the different inverter types

The Data Layer is responsible for the object-oriented data communication between the individual bus participants. The data are filed in objects.

Every data object has a description (data description object) that provides information on type, sort, accuracy and unit of the data object.

The data is accessible similar as described in chapter 3.

Due to the ease of use the data is always sent as readable, volatile data object with mostly fixed precision. An exception is the TL3 inverter that uses different precisions.

The access to these objects is a little more complex than the access to the string objects described before.

There is no guarantee for these objects to be present in future inverters and if present anyway the precision of the objects may change.

Therefore the access to these objects has to be done with caution.

4.2 Reading data

4.2.1 Telegram „Read Data“ and response

This telegram is used to read one of the data objects listed above of a participant.

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0xF2	Telegram type = PTYPE_REQUEST_DATA_RQ
12	1	0x80	Packet number
13	2	...	Data object ID as listed in “Table 28: List of the relevant object IDs”

Table 31: Data request telegram

The respective participant responds to the inquiry with the following telegram:

Offset	Length	Content	Description
3	4	...	32-bit target serial number
7	4	...	32-bit source serial number
11	1	0x72	Telegram type = PTYPE_REQUEST_DATA_AN
12	1	0x80	Packet number
13	n	...	Requested data

Table 32: Data response telegram

The required data is transmitted in the data field of the telegram. The data is delivered as described in “Table 29: Data descriptions of the relevant object IDs”.

5 Appendix A

5.1 Event Codes

Actual event codes can be downloaded from the homepage of the Diehl AKO corporation.

Describe all events below 100 not defined otherwise as "Installation errors" with the corresponding number.
Describe all events below 200 not defined otherwise as "Installation errors" with the corresponding number.
Describe all events below 300 not defined otherwise as "internal system message" with the corresponding number.
Describe all events below 500 not defined otherwise as "Information" with the corresponding number.

Caution: Newer devices will have four digits for the events, using the first three corresponding to the list provided here. Digit four indicates an event code not contained in this specification.

5.2 Event Classes

1	Destructive error, signaled by Display and Buzzer with no delay
4	Blocking error, signaled by Display with no delay
7	Non blocking error, signaled by Display with delay
10	Warning, only stored in the Eventlist
13	Information, not stored

5.3 Event Sources

1	SubSystem UI
2	SubSystem ICU
3	SubSystem GMU
4	3xTL(D) SubSystem ICU L1
5	3xTL(D) SubSystem GMU L1
6	3xTL(D) SubSystem ICU L2
7	3xTL(D) SubSystem GMU L2
8	3xTL(D) SubSystem ICU L3
9	3xTL(D) SubSystem GMU L3
255	3xTL(D) SubSystem UI

5.4 State Codes

0	Inverter standby
1	Inverter waiting for DC voltage
2	Inverter waiting for supply voltage
10-11	Inverter is active (status code: XX)
12	Inverter is active and runs in the MPP
13-29	Inverter is active (status code: XX)
15	Inverter is active and runs in reduced mode (commanded by ENS or due to phase balancing or overtemperature)
16-29	Inverter is active (status code: XX)

30-32	Inverter startup test mode
49-81	Internal modes -> contact service if active
100	Inverter in error state

5.5 Support Matrix for the EasyConnect Objects

This table contains the software releases from which on the easyconnect data objects are available. Older software versions only support the more complex single data objects.

Inverter Type	Software Version, the EasyConnect objects are supported from
PLATINUM S	712107-26
PLATINUM TL	not yet supported, planned for next version 733024-08
PLATINUM 3xTL	not yet supported, planned for next version 738450-03
PLATINUM TL3	739504-03
PLATINUM TLD	745896-02 (Firmware Package 1.1.0.0)
PLATINUM TLxD	748177-03 (Firmware Package 1.2.1.0)
PLATINUM CS	not supported, not planned for the future

Table 33: Support Matrix